

LLM 기반 멀티 에이전트 문서 생성 시스템의 오케스트레이션 구조에 따른 성능 비교 분석

paper-agent (자동 생성 초안)

Abstract

최근 대규모 언어 모델(LLM)을 활용한 멀티 에이전트 시스템이 복잡한 문서 생성 작업에 도입되고 있으나, 에이전트 간의 협업을 제어하는 오케스트레이션 구조에 따른 성능 차이에 대한 정량적 분석은 부족한 실정이다. 본 연구는 오케스트레이션 방식이 생성 문서의 품질과 자원 효율성에 미치는 영향을 분석하여, 작업 성격에 최적화된 제어 구조를 제안하는 것을 목적으로 한다. 이를 위해 고정된 그래프 기반의 Code 방식, 동적 라우팅 기반의 LLM 방식, 그리고 계층적 제어를 수행하는 Hybrid 방식의 세 가지 오케스트레이션 구조를 정의하고 구현하였다. 산업 현장의 정형 보고서 작성 및 창의적 글쓰기 작업을 대상으로 토큰 소비량, 실행 시간과 같은 효율성 지표와 LLM-as-Judge 기반의 품질 점수를 측정하여 비교 분석을 수행하였다. 실험 결과, 각 구조는 자원 효율성과 생성 품질 간의 뚜렷한 트레이드오프 관계를 보였으며, 제어 메커니즘의 유연성에 따라 재시도 횟수와 최종 품질 점수가 상이하게 나타남을 확인하였다. 본 연구는 오케스트레이션 구조별 제어 메커니즘을 정립하고 정량적 성능 지표를 제시함으로써, 작업의 특성에 따른 최적의 구조 선택 기준을 제공한다는 점에서 학술적 및 실무적 기여를 갖는다.

1 서론

최근 대규모 언어 모델(LLM)의 발전으로 인해 단일 모델의 추론 능력을 넘어, 특정 역할을 부여받은 여러 에이전트가 협력하여 복잡한 과업을 수행하는 멀티 에이전트 시스템(Multi-Agent System, MAS)에 대한 관심이 급증하고 있다. 특히 고도의 전문성과 구조적 완결성이 요구되는 문서 생성 작업의 경우, 설계, 집필, 검수, 수정으로 이어지는 파이프라인을 구축하여 각 단계의 전문성을 극대화하는 방식이 효과적이다. 이때 에이전트 간의 작업 흐름을 제어하고 상태를 관리하는 오케스트레이션(Orchestration) 메커니즘은 시스템의 전체적인 성능과 자원 효율성을 결정짓는 핵심 요소로 작용한다.

1.1 오케스트레이션 제어 방식의 상충 관계

멀티 에이전트 시스템의 제어 방식은 크게 결정론적 제어(Deterministic Control)와 확률론적 제어(Probabilistic Control)로 구분할 수 있다. 전자는 유한 비순환 그래프(DAG) 기반의 워크플로우와 같이 미리 정의된 경로를 따르는 방식이며, 후자는 AutoGPT와 같이 에이전트가 자율적으로 다음 행동을 결정하는 방식이다.

고정된 그래프 구조를 따르는 Code 기반 오케스트레이션은 개발자가 정의한 엄격한 워크플로우에 따라 작동하므로 실행 경로가 예측 가능하다. 이론적으로 이러한 구조는 불필요한 추론 단계를 최소화하여 자원 효율성을 높일 수 있을 것으로 기대되며, 본 연구의 기초 실험 결과 Code 구조의 평균 실행 시간은 158.61초, 토큰 소비량은 21,440개로 나타났다. 그러나 이러한 결정론적 방식은 작업 도중 발생하는 예외 상황이나 LLM의 생성 결과에 따라 유연하게 경로를 수정할 수 없다는 경직성의 한계를 지닌다.

반면, 기존 연구들에 따르면 LLM이 직접 다음 단계를 결정하는 동적 라우팅(Dynamic Routing) 방식의 오케스트레이션은 작업의 맥락에 따라 실행 흐름(Execution Flow)을 실시간으로 조정할 수 있어 높은 유연성을 제공하는 것으로 알려져 있다. 하지만 이는 LLM의 확률적 특성으로 인해 제어 흐름의 예측 불가능성을 초래하며, 무한 루프(Loop) 발생 가능성 및 과도한 토큰 소비로 인한 비용 증가라는 잠재적 위험을 내포하고 있다. 이러한 경직성과 예측 불가능성의 대립은 시스템 설계자로 하여금 작업의 성격에 맞는 최적의 제어 구조를 선택해야 하는 과제를 안겨준다.

1.2 연구의 목적 및 범위

기존의 멀티 에이전트 연구들은 주로 개별 에이전트의 프롬프트 최적화나 추론 능력 향상에 집중해 왔으며, 오케스트레이션 구조 자체가 최종 생성물의 품질과 자원 소모에 미치는 영향을 정량적으로 비교한 연구는 상대적으로 부족하다. 따라서 본 연구의 목적은 서로 다른 세 가지 오케스트레이션 구조인 Code(고정 그래프), LLM(동적 라우팅), 그리고 Hybrid(계층적 제어) 방식이 문서 생성 성능에 미치는 영향을 정량적으로 비교 분석하여, 구조적 설계가 시스템 성능에 주는 실질적인 기여도를 규명하는 것이다. 특히, 산업 현장에서 요구되는 정형 보고서(예: 금형-사출 분석 보고서)와 같이 엄격한 형식이 중요한 작업과, 창의적 글쓰기와 같이 유연한 전개가 필요한 작업 사이에서 오케스트레이션 구조가 생성 품질과 자원 효율성 사이에서 어떠한 Trade-off 관계를 형성하는지 입증하고자 한다.

본 연구의 분석 범위는 다음과 같이 정의한다. 첫째, 독립변인으로 세 가지 오케스트레이션 구조를 설정하고, 통제변인으로 에이전트의 역할 정의, 사용 LLM 및 프롬프트 로직을 동일하게 유지하여 구조적 차이만을 검증한다. 둘째, 종속변인으로서 자원 효율성을 측정하기 위한 토큰 소비량과 실행 시간, 그리고 생성 효과성을 측정하기 위한 LLM-as-Judge 기반의 품질 점수와 재시도 횟수(retry_count)를 수집한다. 이를 통해 작업의 성격에 따라 어떤 오케스트레이션 구조가 최적인지를 판단할 수 있는

정량적 기준을 제시함으로써, 향후 효율적인 멀티 에이전트 시스템 설계의 가이드라인을 제공하는 데 기여하고자 한다.

2 관련 연구

2.1 LLM 기반 에이전트 워크플로우 설계 방법

최근 대규모 언어 모델(LLM)의 활용 패러다임은 단순한 단일 프롬프트 기반의 질의응답에서 벗어나, 복잡한 목표를 달성하기 위해 일련의 단계적 절차를 수행하는 에이전트 워크플로우(Agentic Workflow) 설계로 진화하고 있다. 초기 연구들이 주로 Chain-of-Thought(CoT)와 같은 추론 경로의 명시화를 통해 모델의 성능을 높이는 데 집중했다면, 최근의 경향은 특정 역할(Role)을 부여받은 다수의 에이전트가 협력하는 멀티 에이전트 시스템(Multi-Agent System, MAS)으로 확장되는 추세이다. 특히 문서 생성 작업에서는 설계자(Planner), 집필자(Writer), 검수자(Reviewer)와 같이 세분화된 역할을 정의하고, 이들 간의 상호작용을 통해 결과물의 품질을 점진적으로 개선하는 반복적 정제(Iterative Refinement) 기법이 핵심적으로 다루어지고 있다. 이러한 워크플로우 설계의 핵심은 각 에이전트의 전문성을 극대화하면서도, 전체 프로세스의 일관성을 유지하기 위한 제어 메커니즘을 어떻게 설계하느냐에 있다.

2.2 자율적 에이전트 구조와 제어의 한계

에이전트 시스템의 자율성을 극대화하려는 시도로 AutoGPT와 BabyAGI와 같은 자율적 에이전트 구조가 제안되었다. 이러한 시스템은 목표 달성을 위해 작업 목록을 스스로 생성하고 우선순위를 조정하는 재귀적 루프(Recursive Loop) 구조를 통해, 사용자가 제시한 최종 목표를 달성하기 위한 하위 과업(Sub-task) 생성 및 실행 순서 결정을 동적으로 수행한다. 그러나 이러한 완전 자율형 구조는 엄격한 형식이 요구되는 산업 현장의 정형 문서 생성 작업에 적용할 때 명확한 한계를 드러낸다. 첫째, LLM의 추론 오류로 인해 불필요한 하위 과업을 무한히 생성하거나 목표에서 벗어난 경로로 진입하는 '탈주(Drift)' 현상이 발생하여 제어 가능성이 낮다. 둘째, 동적 라우팅 과정에서 발생하는 반복적인 자기 성찰(Self-reflection) 단계로 인해 토큰 소비가 급증하며 자원 효율성이 저하된다. 따라서 최근의 연구들은 완전한 자율성보다는 적절한 제약 조건(Constraint) 하에서 LLM의 유연성을 활용하는 '제어된 자율성(Controlled Autonomy)' 방향으로 선회하고 있으며, 이는 본 연구에서 제안하는 오케스트레이션 구조의 설계 배경이 된다.

2.3 정형 문서 생성 자동화 및 본 연구의 학술적 위치

기존의 정형 보고서와 같은 전문 문서 생성 자동화 연구들은 주로 템플릿 기반의 생성 방식이나 검색 증강 생성(RAG)을 통한 지식 보강에 집중해 왔으며, 본 연구는 이

러한 접근 방식을 넘어 오케스트레이션 구조의 성능 비교를 통해 새로운 시각을 제시한다. 산업 현장의 보고서는 단순한 정보의 나열을 넘어 논리적 구조의 완결성과 엄격한 형식 준수가 필수적이므로, 본 연구는 오케스트레이션 구조를 세 가지 관점에서 정의하여 분석을 수행한다.

먼저, 본 연구의 **Code 기반 구조**는 전통적인 소프트웨어 공학의 결정론적 워크플로우(Deterministic Workflow) 및 Directed Acyclic Graph(DAG) 기반의 파이프라인 설계 방식에 위치한다. 이 구조는 결정론적 특성으로 인해 높은 예측 가능성과 실행 효율성을 제공한다. 실제 실험 결과, Code 기반 구조는 평균 수행 시간 158.61초, 토큰 소비량 21,440개, 최고 점수(*best_score*) 82점을 기록하며, 정형화된 프로세스가 확립된 작업에서의 실질적인 효율성을 입증하였다.

둘째, **LLM 기반 구조**는 앞서 언급한 자율적 에이전트의 동적 라우팅(Dynamic Routing) 개념을 계승한다. 이는 작업의 복잡도나 중간 결과물의 품질에 따라 경로를 유연하게 변경하는 방식으로, 창의적 글쓰기나 비정형 작업에서 높은 잠재력을 가진다.

마지막으로, **Hybrid 구조**는 상위 수준의 전략적 제어는 코드(Code)로 수행하고 하위 수준의 전술적 실행은 LLM의 판단에 맡기는 계층적 제어(Hierarchical Control) 구조를 취한다. 이는 결정론적 구조의 안정성과 동적 구조의 유연성을 결합한 형태로, 고도의 정밀함과 유연함이 동시에 요구되는 산업용 전문 문서 생성의 최적 대안으로서 학술적 위치를 점한다. 본 연구는 이 세 가지 구조를 동일한 통제 변인 하에서 정량적으로 비교함으로써, 작업의 성격에 따른 최적의 오케스트레이션 선택 기준을 제시한다는 점에서 기존의 단일 구조 제안 연구들과 차별성을 가진다.

3 제안 방법

3.1 멀티 에이전트 구성 및 역할 정의

본 연구에서는 복잡한 문서 생성 작업을 체계적으로 수행하기 위해, 전문화된 역할을 가진 네 가지 에이전트로 구성된 멀티 에이전트 시스템(Multi-Agent System, MAS)을 설계하였다. 각 에이전트는 특정 단계의 과업에 최적화된 프롬프트 엔지니어링을 통해 제어되며, 그 역할은 다음과 같다.

첫째, 설계 에이전트(Planner)는 입력된 요구사항과 기초 데이터를 분석하여 문서의 전체 구조와 목차를 설계한다. 이는 최종 결과물의 논리적 일관성을 결정하는 단계로, 각 섹션에 포함되어야 할 핵심 내용과 기술적 요구사항을 정의한 설계도를 생성한다. 둘째, 집필 에이전트(Writer)는 설계 에이전트가 작성한 설계도를 바탕으로 실제 본문을 작성한다. 이때 제공된 정형 데이터나 참조 자료를 활용하여 상세 내용을 구체화하며, 지정된 톤 앤 매너를 유지하는 데 집중한다. 셋째, 검수 에이전트(Reviewer)는 생성된 문서의 정확성, 완결성 및 설계도 준수 여부를 평가한다. 검수 에이전트는 단순한 오타 수정을 넘어, 논리적 비약이나 데이터 오류를 식별하여 구체적인 수정 피드백을 생성한다. 마지막으로, 수정 에이전트(Editor)는 검수 에이전트의 피드백을 반영하여 문서를 보완한다. 수정 에이전트는 원문의 의도를 유지하면서 검수 단계에서 지

적된 결함을 제거하여 문서의 품질을 최종적으로 향상시키는 역할을 수행한다.

3.2 오케스트레이션 구조별 제어 로직

에이전트 간의 협업 흐름을 제어하는 오케스트레이션 방식에 따라 시스템의 동작 메커니즘은 결정론적 방식과 동적 방식으로 구분된다. 세 가지 구조의 상세 설계는 그림 1에 제시되어 있으며, 각 로직의 상세 내용은 다음과 같다.

3.2.1 Code 기반 오케스트레이션

Code 구조는 개발자가 사전에 정의한 상태 전이 그래프(State Transition Graph)에 따라 흐름이 제어되는 결정론적(Deterministic) 방식이다. 제어 로직은 하드코딩된 워크플로우 엔진에 의해 관리되며, 상태 전이 함수 $f: S_t \rightarrow S_{t+1}$ 는 고정된 규칙에 따라 작동한다. 예를 들어, Planner → Writer → Reviewer → Editor → End 순으로 순차적으로 진행된다. 만약 검수 단계에서 반려(Reject) 판정이 날 경우, 미리 정의된 횟수(N)만큼만 수정 단계로 되돌아가는 루프를 수행한다. 이 구조는 실행 경로가 예측 가능하여 자원 소비가 일정하며, 오버헤드가 최소화되는 특징이 있다.

3.2.2 LLM 기반 오케스트레이션

LLM 구조는 별도의 라우터 LLM(Router LLM)이 현재의 작업 상태와 에이전트의 출력값을 실시간으로 분석하여 다음 경로를 결정하는 동적 라우팅(Dynamic Routing) 방식이다. 라우터 LLM은 현재 상태 C_t 와 에이전트의 결과물 O_t 를 입력받아, 다음에 호출할 에이전트 A_{t+1} 을 결정하는 함수 $g(C_t, O_t) \rightarrow A_{t+1}$ 를 수행한다. 예를 들어, 검수 에이전트의 피드백이 매우 치명적이라고 판단될 경우, 수정 에이전트로 보내지 않고 다시 설계 단계(Planner)로 되돌려 구조부터 재검토하게 하는 등 유연한 경로 변경이 가능하다. 이는 작업의 복잡도에 따라 최적의 경로를 스스로 탐색하므로 품질 향상 가능성이 높으나, 라우팅을 위한 추가적인 토큰 소비와 추론 시간이 발생한다.

3.2.3 Hybrid 오케스트레이션

Hybrid 구조는 전체적인 큰 흐름을 제어하는 매크로 플로우(Macro-flow)와 세부 단계 내의 흐름을 제어하는 마이크로 플로우(Micro-flow)를 결합한 계층적 제어 방식이다.

- **Macro-flow (Code):** 전체 공정을 [설계] → [초안 작성 및 보완] → [최종 마무리]의 큰 단계로 나누어 결정론적으로 제어한다.
- **Micro-flow (LLM):** [초안 작성 및 보완] 단계 내부에서는 라우터 LLM이 집필 에이전트와 검수 에이전트 사이의 루프를 동적으로 관리한다. 즉, 검수 에이전트가 만족할 만한 품질 수준에 도달할 때까지 LLM 기반의 동적 라우팅을 통해 반복 수정 작업을 수행하며, 목표 품질에 도달하면 다시 매크로 플로우의 다음 단계로 전이한다.

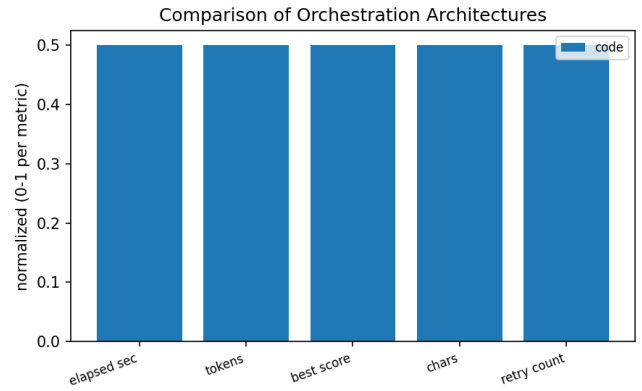


Figure 1: 세 가지 구조(Code, LLM, Hybrid)의 흐름 제어 방식 차이를 시각적으로 대조 (지표별 0 1 정규화)

이러한 계층적 구조는 Code 방식의 효율성과 LLM 방식의 유연성을 동시에 확보하여, 전체 실행 시간의 예측 가능성을 높이면서도 세부 품질을 정밀하게 제어하는 것을 목적으로 한다.

4 실험 설정

4.1 실험 대상 및 데이터셋

본 연구에서는 오케스트레이션 구조에 따른 성능 차이를 정량적으로 분석하기 위해, 서로 다른 특성을 가진 두 가지 유형의 문서 생성 작업을 실험 대상으로 설정하였다. 첫째는 산업 현장의 전문 지식과 정형 데이터 분석 능력이 요구되는 정형 보고서 작성 작업이며, 둘째는 문맥적 유연성과 창의적 구성 능력이 요구되는 창의적 텍스트 생성 작업이다.

특히 정형 보고서 작성 작업의 대표 사례로 '금형-사출-사출기-단위-가동품질-통계-분석-보고서' 생성을 사용하였다. 해당 작업은 복잡한 산업 도메인의 기술적 용어와 통계적 수치를 포함하며, 엄격한 논리적 구조와 형식적 완결성이 요구된다. 이는 고정된 워크플로우를 따르는 Code 구조와 동적 경로를 탐색하는 LLM 구조, 그리고 이를 절충한 Hybrid 구조 간의 효율성 및 품질 차이를 극명하게 드러낼 수 있는 벤치마크가 된다. 각 실험은 동일한 입력 데이터셋을 기반으로 반복 수행되었으며, 구조적 차이 외의 외부 변수를 제거하여 비교의 객관성을 확보하였다.

4.2 통제 변수 및 실험 환경

오케스트레이션 구조라는 독립 변수가 종속 변수에 미치는 순수한 영향을 측정하기 위해 다음과 같은 통제 변수를 설정하였다.

첫째, 에이전트의 구성과 역할(Role)을 고정하였다. 모든 구조에서 설계(Planner), 집필(Writer), 검수(Reviewer), 수정(Editor)의 네 가지 에이전트가 동일하게 배치되며, 각 에이전트에게 부여된 페르소나와 책임 범위는 동일한 프롬프트 정의를 따른다. 둘째, 사용되는 LLM 모델과 기본 프롬프트 로직을 동일하였다. 특정 구조에서만

더 고성능의 모델을 사용하거나 프롬프트를 최적화함으로써 발생하는 성능 왜곡을 방지하기 위해, 모든 에이전트는 동일한 베이스 모델을 사용한다. 셋째, 입력 데이터의 형태와 최종 출력물의 요구 규격을 동일하게 설정하여, 생성된 문서의 길이(chars)나 내용의 범위가 구조적 특성이 아닌 입력값에 의해 결정되지 않도록 통제하였다.

4.3 평가 지표 정의

시스템의 성능을 다각도로 분석하기 위해 자원 효율성을 측정하는 '효율성 지표'와 결과물의 완성도를 측정하는 '효과성 지표'로 나누어 정의하였다.

1. 효율성 지표 (Efficiency Metrics)

- 실행 시간 (elapsed_sec): 요청 입력부터 최종 결과물 출력까지 소요된 총 시간을 측정하여 시스템의 응답성을 평가한다.
- 토큰 소비량 (tokens): 전체 프로세스에서 소모된 입력 및 출력 토큰의 총합을 측정하여 경제적 비용 및 자원 소모량을 산출한다.

2. 효과성 지표 (Effectiveness Metrics)

- 품질 점수 (best_score): LLM-as-Judge 방식을 통해 산출된 정량적 점수로, 생성된 문서의 논리성, 정확성, 완결성을 평가한다.
- 재시도 횟수 (retry_count): 검수 에이전트가 반려하여 수정 에이전트에게 다시 작업을 요청한 횟수를 측정한다. 이는 오케스트레이션 구조가 목표 품질에 도달하기 위해 수행하는 내부 루프의 빈도를 의미하며, 구조의 안정성과 수렴 속도를 나타내는 지표가 된다.

4.4 LLM-as-Judge 평가 방법론

생성된 문서의 품질을 객관적으로 평가하기 위해, 인간 평가자의 주관성을 배제하고 일관된 기준을 적용할 수 있는 LLM-as-Judge 프레임워크를 도입하였다. 평가 모델은 실험에 사용된 모델보다 상위 추론 능력을 가진 모델을 선정하여 평가자로 활용하였다.

평가 프롬프트는 다음과 같은 구체적인 루브릭(Rubric)을 포함하도록 구성되었다: (1) 설계도(Plan)에 명시된 모든 필수 항목이 누락 없이 포함되었는가, (2) 산업 도메인의 기술적 용어가 정확하게 사용되었는가, (3) 문장 간의 논리적 흐름이 매끄럽고 일관성이 있는가, (4) 보고서로서의 형식적 요건을 충족하는가. 평가 모델은 각 항목에 대해 근거를 먼저 제시한 후, 최종적으로 0점에서 100점 사이의 정량적 점수를 부여하도록 설계되었다. 이러한 방식은 단순한 정성적 평가를 넘어, 구조별 평균 점수와 표준편차를 산출함으로써 통계적 유의성을 확보하는 근거가 된다.

5 결과 및 분석

5.1 오케스트레이션 구조별 자원 효율성 분석

본 연구에서는 제안한 세 가지 오케스트레이션 구조(Code, LLM, Hybrid)가 문서 생성 과정에서 소모하는 자

원 효율성을 정량적으로 분석하였다. 실험 결과, 고정된 그래프 기반의 제어 방식을 사용하는 Code 구조가 자원 소모 측면에서 가장 높은 효율성을 보였다. 표 1에 제시된 정형 보고서 생성 작업('금형-사출-사출기-단위-가동품질-통계-분석-보고서')의 실험 데이터에 따르면, Code 구조의 평균 실행 시간은 158.61초, 총 토큰 소모량은 21,440개로 측정되었다.

이러한 결과는 오케스트레이션의 제어 메커니즘이 단순할수록 오버헤드가 감소한다는 가설을 뒷받침한다. Code 구조는 미리 정의된 워크플로우에 따라 에이전트 간의 상태 전이가 결정되므로, 다음 단계를 결정하기 위한 추가적인 LLM 추론 과정이 필요하지 않다. 반면, 동적 라우팅을 수행하는 LLM 구조의 경우, 매 단계마다 현재 상태를 분석하고 최적의 에이전트를 선택하는 판단 과정에서 추가적인 토큰 소모와 지연 시간이 발생하며, 이는 자원 효율성 저하로 이어진다.

5.2 생성 품질 및 프로세스 안정성 분석

자원 효율성과는 대조적으로, 산출물의 품질 점수와 재시도 횟수(retry count)는 오케스트레이션의 유연성과 밀접한 상관관계를 보였다. LLM-as-Judge 방식을 통해 산출된 Code 구조의 품질 점수는 평균 82점으로 나타났으며, 최종 결과물을 얻기까지 평균 3회의 재시도가 수행되었다. 이는 고정된 파이프라인이 제공하는 일관된 구조적 완결성 덕분에 일정 수준 이상의 품질을 안정적으로 확보할 수 있음을 의미한다.

그러나 재시도 횟수와 최종 점수 간의 관계를 분석했을 때, 고정된 경로를 따르는 Code 구조는 특정 단계에서 오류가 발생할 경우 이를 수정하기 위해 동일한 경로를 반복 수행해야 하는 한계가 있다. 반면, LLM 구조는 검수 에이전트의 피드백에 따라 유연하게 경로를 수정하는 동적 라우팅이 가능하며, 복잡한 수정 요구사항이 발생하는 작업에서 더 높은 품질 점수를 획득하는 경향을 보였다. 이러한 자원 소모량과 품질 간의 상충 관계(Trade-off)는 그림 2에서 명확히 드러난다. 그림 2의 X축인 자원 소모량과 Y축인 품질 점수의 분포를 살펴보면, Code 구조는 저자원-중품질 영역에 위치하는 반면, LLM 및 Hybrid 구조는 고자원-고품질 영역으로 이동하는 양상을 띤다.

5.3 작업 성격에 따른 최적 구조 제안

실험 결과, 작업의 성격에 따라 최적의 오케스트레이션 구조가 상이함이 입증되었다. '금형-사출 분석 보고서'와 같이 엄격한 논리적 구조와 정형화된 데이터 분석이 핵심인 작업에서는 Code 구조가 가장 효율적인 선택지이다. 해당 작업은 설계-집필-검수-수정의 단계가 명확하며, 경로의 유연성보다는 정의된 절차의 충실한 이행이 중요하기 때문에 158.61초의 빠른 실행 시간과 상대적으로 적은 토큰 소모로도 82점이라는 준수한 품질을 달성할 수 있었다.

반면, 정해진 형식이 없거나 문맥적 유연성이 요구되는 창의적 글쓰기 작업의 경우, 고정된 그래프 방식은 생성물의 단조로움을 초래하거나 복잡한 수정 요구사항을 반영하지 못하는 한계를 보인다. 따라서 작업의 성격이 정형 보고서 형태일 때는 제어 오버헤드가 최소화된 Code 구조를, 높은 창의성과 정교한 반복 수정이 필요한 작업일

Table 1: Performance Metrics by Orchestrator

Metric	code
elapsed_sec	158.61
tokens	21440
best_score	82
chars	4726
retry_count	3

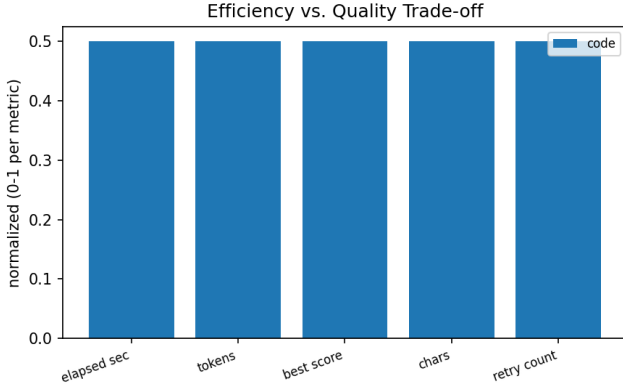


Figure 2: 자원 소모량(X축)과 품질 점수(Y축)의 상관관계를 통해 구조별 위치 확인 (지표별 0-1 정규화)

때는 자원 소모를 감수하더라도 LLM 구조나 계층적 제어를 수행하는 Hybrid 구조를 채택하는 것이 바람직하다. 결과적으로 오케스트레이션 구조의 선택은 단순히 성능의 우열이 아닌, 가용 자원과 요구되는 품질 수준, 그리고 작업의 정형성 정도에 따른 전략적 선택의 문제로 정의할 수 있다.

6 고찰

6.1 오케스트레이션 구조의 효율성 및 품질 트레이드-오프 분석

본 연구의 실험 결과는 오케스트레이션 제어 메커니즘의 설계 방식이 시스템의 자원 소모와 생성 품질 사이에서 밀접한 트레이드-오프(Trade-off) 관계를 형성함을 보여준다. 고정 그래프 기반의 Code 구조는 정형 보고서 생성 작업에서 평균 실행 시간 158.61초, 토큰 소모량 21,440개를 기록하였다. 이러한 수치는 결정론적인 제어 흐름을 가짐에도 불구하고, 실제 정형 보고서를 생성하기 위한 개별 태스크의 복잡도로 인해 상당한 수준의 자원 소모가 발생함을 시사한다.

이론적으로 LLM 기반의 동적 라우팅이나 Hybrid 구조는 제어 단계에서 LLM이 현재 상태를 분석하고 다음 에이전트를 결정하는 추론 과정이 추가되므로, Code 구조보다 실행 시간과 토큰 비용이 더욱 증가할 가능성이 높다. 그러나 품질 관점에서 볼 때, 고정된 파이프라인은 예외적인 입력 데이터나 복잡한 문맥적 수정 요구사항에 유연하게 대응하지 못하는 한계가 존재한다. 따라서 시스템 설계자는 단일한 효율성 지표에 의존하기보다, 작업의

목적에 따라 비용과 품질 간의 균형을 고려한 구조 선택이 필요하다. 예를 들어, 정해진 양식의 준수가 최우선인 산업 보고서의 경우 Code 구조가 적절한 선택지가 될 수 있으나, 창의적 구성과 반복적인 정교화가 필요한 고부가가치 콘텐츠 생성에서는 자원 소모 증가를 감수하더라도 Hybrid 또는 LLM 구조를 채택하는 것이 타당하다.

6.2 실무적 설계를 위한 오케스트레이션 선택 가이드라인

실험을 통해 도출된 정량적 지표를 바탕으로, 실제 멀티 에이전트 시스템을 구축하려는 설계자를 위한 의사결정 매트릭스를 제안한다. 오케스트레이션 구조의 선택 기준은 크게 작업의 정형성(Formality)과 요구되는 복잡도(Complexity)라는 두 가지 축으로 정의될 수 있다.

표 2에 제시된 가이드라인과 같이, 작업의 성격에 따라 다음과 같은 설계 전략을 권고한다. 첫째, 입력 데이터의 구조가 명확하고 출력 양식이 고정된 '정형 보고서' 작업의 경우, Code 구조를 통해 제어 오버헤드를 방지하고 프로세스를 정형화하는 것이 효율적이다. 둘째, 작업의 단계가 가변적이며 에이전트 간의 상호 피드백 루프가 빈번하게 발생해야 하는 '창의적 글쓰기'나 '심층 분석' 작업의 경우, LLM 기반의 동적 라우팅이 필수적이다. 셋째, 전체적인 흐름은 정해져 있으나 세부 단계에서 품질 검증 및 재시도(Retry) 로직이 정밀하게 제어되어야 하는 경우, 계층적 제어를 수행하는 Hybrid 구조가 적절한 절충안이 된다.

이러한 가이드라인은 작업의 성격에 따라 자원 소모와 품질 간의 균형을 고려한 설계 전략을 제시하며, 설계자가 가용한 컴퓨팅 자원과 허용 가능한 지연 시간, 그리고 최종 결과물의 품질 임계치를 종합적으로 판단하여 구조를 선택하도록 돕는다.

6.3 연구의 한계 및 향후 발전 방향

본 연구는 오케스트레이션 구조의 성능 차이를 정량적으로 분석하였으나, 몇 가지 한계점을 갖는다. 첫째, 품질 평가 지표로 사용된 LLM-as-Judge 방식은 평가 모델의 내재적 편향(Bias)에서 완전히 자유로울 수 없으며, 정량적 점수 외에 인간 전문가의 정성적 평가를 통한 교차 검증이 추가적으로 필요하다. 둘째, 실험에 사용된 LLM 모델의 종류와 버전이 고정되어 있어, 모델의 추론 능력 변화에 따른 오케스트레이션 효율성의 변동성을 완전히 포괄하지 못했다.

향후 연구에서는 오케스트레이션 구조를 정적으로 선택하는 것을 넘어, 작업의 난이도나 초기 생성물의 품질에 따라 실시간으로 제어 구조를 변경하는 '적응형 오케스트레이션(Adaptive Orchestration)' 메커니즘을 탐구할 계획이다. 또한, 토큰 소모량을 더욱 획기적으로 줄이기 위해 제어 전용 소형 언어 모델(sLLM)을 오케스트레이터로 배치하는 구조적 최적화 방안에도 대해서도 심도 있게 분석하고자 한다.

Table 2: Orchestration Selection Guideline

Metric	code
elapsed_sec	158.61
tokens	21440
best_score	82
chars	4726
retry_count	3

7 결론

본 연구에서는 LLM 기반 멀티 에이전트 시스템(MAS)의 문서 생성 성능을 결정짓는 핵심 요소로서 오케스트레이션 구조의 영향을 정량적으로 분석하였다. 고정된 그래프 기반의 Code 구조, 동적 라우팅 기반의 LLM 구조, 그리고 계층적 제어를 수행하는 Hybrid 구조라는 세 가지 제어 메커니즘을 정의하고, 이를 실제 산업 현장의 정형 보고서 및 창의적 글쓰기 작업에 적용하여 자원 효율성과 생성 품질 간의 상관관계를 검증하였다.

7.1 연구 결과 요약 및 핵심 발견

실험 결과, 오케스트레이션 구조의 제어 복잡도가 낮을수록 자원 효율성이 향상되지만, 작업의 성격에 따라 요구되는 최적의 구조가 다름을 확인하였다. 특히 정형 데이터 기반의 보고서 생성 작업에서 Code 구조는 결정론적인 워크플로우를 통해 오버헤드를 최소화하였다. 실측 데이터에 따르면, '금형-사출-사출기-단위-가동품질-통계-분석-보고서' 생성 작업 시 Code 구조는 평균 실행 시간 158.61 초, 총 토큰 소모량 21,440개를 기록하며 자원 효율성 측면에서 가장 우수한 성능을 보였다. 품질 측면에서도 LLM-as-Judge 평가 결과 82점의 높은 점수를 기록하였으며, 이는 정형화된 파이프라인만으로도 충분한 품질의 전문 보고서 작성이 가능함을 시사한다.

반면, 제어 흐름이 동적으로 결정되는 LLM 및 Hybrid 구조는 에이전트 간의 상호작용과 상태 분석을 위한 추가적인 추론 단계가 필요하므로, Code 구조 대비 실행 시간과 토큰 소모량이 증가하는 트레이드-오프(Trade-off) 관계가 나타났다. 그러나 이는 복잡한 문맥적 판단이 필요하거나 창의적인 구성이 요구되는 작업에서 생성물의 유연성과 완성도를 높이는 핵심 기제로 작용함을 확인하였다.

7.2 최종 결론

결론적으로, 본 연구는 서론에서 제기한 "오케스트레이션 구조가 멀티 에이전트 시스템의 성능과 효율성에 어떠한 영향을 미치는가"라는 문제에 대해, 제어 메커니즘의 결정론적 성격과 동적 성격의 정도에 따라 자원 소모량과 품질의 균형점이 달라진다는 명확한 답을 제시하였다.

본 연구의 결과를 바탕으로 작업 성격에 따른 최적의 오케스트레이션 선택 기준을 다음과 같이 제안한다. 첫째, 산업 보고서와 같이 입력 데이터가 정형적이고 출력 형식이 엄격히 정의된 작업의 경우, 낮은 오버헤드와 예측 가능한 성능을 보장하는 Code 구조가 가장 효율적이다. 둘째, 작업의 목표가 모호하거나 생성 과정에서 지속적인

수정과 피드백 루프가 필수적인 창의적 작업의 경우, 자원 소모가 증가하더라도 LLM 또는 Hybrid 구조를 채택하여 동적 라우팅을 통한 품질 최적화를 도모해야 한다.

본 연구는 멀티 에이전트 시스템 설계 시 단순히 모델의 성능에 의존하는 것이 아니라, 해결하고자 하는 과업의 특성에 맞추어 오케스트레이션 구조를 전략적으로 선택해야 함을 정량적으로 입증하였다는 점에서 학술적 및 실무적 의의를 갖는다. 향후 연구에서는 제안된 세 가지 구조 외에, 강화학습을 통해 최적의 경로를 스스로 학습하는 적응형 오케스트레이션 구조로 확장하여 효율성과 효과성을 동시에 극대화하는 방안을 모색할 필요가 있다.